

EduSaaS Adaptive Quiz & Skill Gap Analysis

Complete Technical Documentation

Node.js / Express + Flask / Python + PostgreSQL

1. Documentation Scope

This document is based on the uploaded Node.js and Python source files. It documents the current Adaptive Quiz and Skill Gap architecture, API routes, function calls, adaptive logic, scoring, database flow, and the relationship between the two modules.

The Python AdaptiveEngine is explicitly designed as a pure logic layer with no Flask or database dependencies. The Node.js layer is responsible for database access and persistence, while Flask route modules adapt HTTP requests into engine calls.

2. Project Structure

```
C:\Adaptive_Quiz
├── backend
│   ├── app.js
│   ├── server.js
│   ├── package.json
│   ├── package-lock.json
│   ├── testpython.js
│   ├── testSkillGap.js
│   ├── config\db.js
│   ├── controllers
│   │   ├── quizController.js
│   │   └── skillGapController.js
│   ├── routes
│   │   ├── quiz.js
│   │   └── skillGap.js
│   └── services
│       ├── pythonService.js
│       └── skillGapPythonService.js
├── routes
│   ├── quiz.py
│   └── skill_gap.py
└── services
    ├── adaptive_engine.py
    └── skill_gap_engine.py
```

3. File-by-File Documentation

backend/app.js — Creates Express, enables CORS/JSON middleware, and mounts /api/quiz and /api/skill-gap.

backend/server.js — Loads dotenv, imports the Express app, selects PORT or 3000, and starts the server.

backend/config/db.js — Provides the PostgreSQL pool used by Node.js database operations.

backend/controllers/quizController.js — Orchestrates assessment sessions, skills, questions, Python calls, quiz state, scoring, skill transitions, and completion.

backend/controllers/skillGapController.js — Handles Node.js Skill Gap requests and delegates processing to the Skill Gap Python service.

backend/routes/quiz.js — Defines the Node.js Quiz API routes.

backend/routes/skillGap.js — Defines the Node.js Skill Gap API routes.

backend/services/pythonService.js — Node-to-Python integration for quiz state, question selection, answer submission, and scoring.

backend/services/skillGapPythonService.js — Node-to-Python integration for Skill Gap analysis.

routes/quiz.py — Flask Quiz API exposing create-state, next-question, submit-answer, calculate-score, next-skill, and finish.

routes/skill_gap.py — Flask Skill Gap API exposing POST /analyze.

services/adaptive_engine.py — Pure adaptive logic for state, difficulty, question selection, answers, scoring, next skill, and completion.

services/skill_gap_engine.py — Compares student skill levels with required levels and returns gaps, readiness, and missing skills.

backend/testpython.js — Tests Python quiz-service state creation.

backend/testSkillGap.js — Tests Skill Gap analysis using sample Python and SQL levels.

4. Overall Architecture

```
Frontend
|
| HTTP / JSON
v
Node.js / Express
|
+--> Quiz Controller --> pythonService --> Flask quiz.py --> AdaptiveEngine
|
+--> Skill Gap Controller --> skillGapPythonService --> Flask skill_gap.py -->
SkillGapEngine
|
v
PostgreSQL
```

Node.js is the orchestration and persistence layer. Flask provides Python HTTP endpoints. The Python engines contain the assessment and Skill Gap business logic.

5. Express API Layer

```
app.use(cors());
app.use(express.json());
app.use("/api/quiz", quizRoutes);
app.use("/api/skill-gap", skillGapRoutes);
```

The application exposes the Quiz API under /api/quiz and Skill Gap API under /api/skill-gap.

6. Quiz Python API — quiz.py

quiz.py creates a Flask Blueprint and one AdaptiveEngine instance. Each endpoint validates JSON input and calls exactly one corresponding AdaptiveEngine method.

- POST /create-state → engine.create_quiz_state(session_id, skill).

- POST /next-question → engine.get_next_question(state, questions).
- POST /submit-answer → engine.submit_answer(state, question, selected_option).
- POST /calculate-score → engine.calculate_skill_score(state).
- POST /next-skill → engine.get_next_skill(skills, current_skill_index).
- POST /finish → engine.finish_quiz(state).

7. AdaptiveEngine Configuration

```
EASY = 1
MEDIUM = 2
HARD = 3

MAX_QUESTIONS = 10

DIFFICULTY_MARKS:
    Easy = 1
    Medium = 2
    Hard = 3
```

Each skill has a maximum of 10 questions. Marks increase with difficulty.

8. Initial Quiz State

```
{
    session_id,
    skill_id,
    skill_name,
    current_difficulty: 1,
    correct_streak: 0,
    wrong_streak: 0,
    questions_answered: 0,
    obtained_score: 0,
    maximum_score: 0,
    asked_questions: []
}
```

Every skill starts at Easy with no previous answers or score.

9. Adaptive Difficulty

```
Correct answer:
    correct_streak += 1
    wrong_streak = 0
```

```
Wrong answer:
    wrong_streak += 1
    correct_streak = 0
```

```
2 correct answers:
    Easy -> Medium
    Medium -> Hard
```

```
2 wrong answers:
    Hard -> Medium
    Medium -> Easy
```

Difficulty is bounded between Easy and Hard. The relevant streak is reset when a difficulty transition occurs.

10. Question Selection

`get_next_question` filters questions by the current difficulty and excludes question IDs already in `asked_questions`. If no question is available at the current difficulty, it falls back to any unanswered question. It then randomly chooses one.

```
current_difficulty
|
v
filter_matching_questions
|
remove_asked_questions
|
if_empty -> fallback_to_any_unanswered_question
|
random.choice(...)
|
next_question
```

11. Answer Processing

`submit_answer` compares `selected_option` with `correct_option` after converting the selected answer to uppercase. Marks are based on the current difficulty. The question count and maximum score are incremented; `obtained_score` increases only when the answer is correct.

```
is_correct = selected_option.upper() == correct_option.upper()

marks = DIFFICULTY_MARKS[current_difficulty]

questions_answered += 1
maximum_score += marks

if is_correct:
    obtained_score += marks

update_difficulty(...)
```

The result contains correctness, marks awarded, current difficulty, skill completion status, and the updated state.

12. Skill Completion and Scoring

A skill is completed when `questions_answered` reaches 10.

```
percentage = (obtained_score / maximum_score) * 100

>= 90 -> level 5
>= 70 -> level 4
>= 50 -> level 3
>= 25 -> level 2
< 25 -> level 1
```

`calculate_skill_score` returns the session, skill, question count, obtained score, maximum score, percentage, skill level, and Completed status.

13. Next Skill and Finish

`get_next_skill` increments `current_skill_index`. If there is no next skill it returns `None`; otherwise it returns the next index and skill.

`finish_quiz` returns `assessment_completed=true` and the completed skill score.

14. Node.js Adaptive Quiz Workflow

1. Student starts assessment.
2. `quizController` validates `user_id` and `domain_role_id`.
3. Node.js creates `education.quiz_sessions`.
4. Node.js finds the first required skill.
5. Node.js loads active questions.
6. `pythonService` calls `/create-state`.
7. `AdaptiveEngine` creates initial state.
8. Node.js stores `quiz_state`.
9. `pythonService` calls `/next-question`.
10. `AdaptiveEngine` selects a question.
11. Node.js removes `correct_option` from the client response.
12. Student submits an answer.
13. Node.js loads the question and quiz state.
14. `pythonService` calls `/submit-answer`.
15. `AdaptiveEngine` evaluates and adapts difficulty.
16. Node.js saves updated quiz state.
17. After 10 questions, calculate skill score.
18. Save `student_skill_results`.
19. Start the next required skill.
20. After all skills, complete the assessment.

15. Question Answer Security

The Node.js controller internally reads `correct_option` but removes it before returning a question to the frontend. This prevents the correct answer from being exposed to the student.

16. Skill Gap API — `skill_gap.py`

`skill_gap.py` creates a `SkillGapEngine` instance and exposes POST `/analyze`. It requires `student_skills` and `required_skills` and passes both arrays to `analyze_gap`.

17. SkillGapEngine Algorithm

```
student_map[skill_id] = student skill
```

```
For every required skill:
```

```
    student_level = mapped level or 0  
    gap = required_level - student_level  
    if gap < 0: gap = 0
```

```
gap == 0:
```

```
    Ready
```

```
otherwise:
```

```
    Needs Improvement
```

```
    add skill to missing_skills
```

For every required skill, the report contains skill_id, skill_name, required_level, student_level, gap, and status.

18. Skill Gap Readiness

```
total_student = sum(student_levels)
total_required = sum(required_levels)

readiness_score =
    (total_student / total_required) * 100
```

The score is rounded to two decimal places. If total_required is zero, readiness_score is zero.

19. Skill Gap Output

```
{
  "skill_gap": [...],
  "readiness_score": number,
  "missing_skills": [...]
}
```

The skill_gap array contains the per-skill comparison. missing_skills contains skills whose gap is greater than zero.

20. Example

```
Student:
  Python = 3
  SQL    = 2

Required:
  Python = 5
  SQL    = 3

Python gap = 2
SQL gap    = 1

Total student = 5
Total required = 8

Readiness = 62.50%
```

21. End-to-End Skill Gap Workflow

```
Adaptive Quiz
  |
  v
student_skill_results
  |
  v
Node.js Skill Gap API
  |
  v
skillGapPythonService
  |
  v
```

```

Flask /analyze
|
v
SkillGapEngine.analyze_gap()
|
+--> compare levels
+--> calculate gaps
+--> identify missing skills
+--> calculate readiness
|
v
Skill Gap Result

```

22. Database Flow

```

quiz_sessions
|
v
domain_required_skills
|
v
skills
|
v
questions
|
v
quiz_state
|
v
student_answers
|
v
student_skill_results

```

The Node.js controller is responsible for database persistence, while the active AdaptiveEngine class is database-independent.

23. Important Architecture Distinction

The uploaded `adaptive_engine.py` contains the active pure AdaptiveEngine class at the top. Later in the same file there is an older database-oriented implementation inside a triple-quoted block. That older section contains functions such as `start_quiz`, `start_skill`, `get_next_question`, `submit_answer`, `update_difficulty`, `calculate_skill_score`, and `finish_quiz`.

For the current documented architecture, the active AdaptiveEngine should be treated as the Python business-logic layer, while Node.js remains responsible for database operations and persistence.

24. Testing

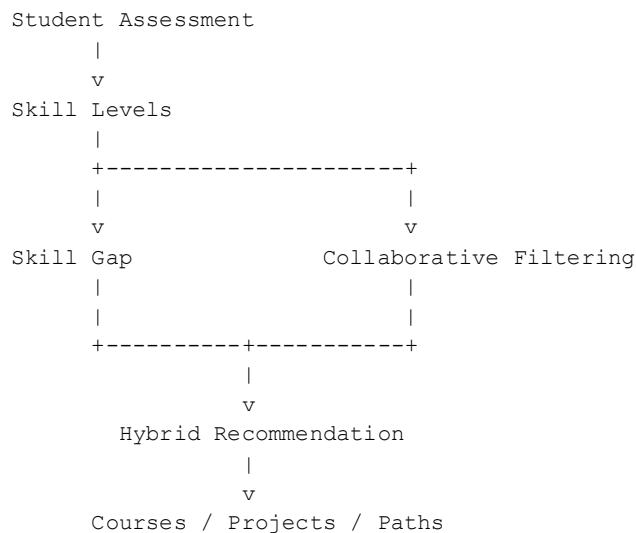
- `testpython.js` calls `pythonService.createState` with a sample session and Python skill.
- `testSkillGap.js` calls `skillGapService.analyzeGap` using Python level 3, SQL level 2, required Python level 5, and required SQL level 3.

These tests validate Node-to-Python service communication and expected input structures.

25. Technologies Used

- Node.js — backend runtime.
- Express.js — API server and routing.
- Flask — Python HTTP API layer.
- Python — AdaptiveEngine and SkillGapEngine.
- PostgreSQL — persistent database.
- JavaScript — Node.js controllers, routes, and services.
- CORS — cross-origin request support.
- dotenv — environment configuration.
- HTTP/JSON — communication between application layers.

26. Future Recommendation Engine Integration



The current uploaded files implement Adaptive Quiz and Skill Gap Analysis. A collaborative-filtering or hybrid recommendation engine is not present in these files. The natural integration point is after student skill results and skill-gap results are available.

27. Final Summary

EduSaaS uses a layered architecture in which Node.js/Express manages API orchestration and PostgreSQL persistence, Flask exposes Python operations, AdaptiveEngine implements adaptive assessment logic, and SkillGapEngine calculates job-role skill gaps.

The complete assessment flow is: select role → identify required skills → initialize skill state → select adaptive questions → submit answers → update difficulty → complete ten questions → calculate skill level → move to next skill → complete assessment → analyze skill gaps. These outputs provide the foundation for the planned recommendation engine.